

10-Missing Integer Project

First algorithm (non-recursive) :

Idea: Create a boolean array to record which positive numbers are present, then return the smallest positive number that is not marked.

-Pseudocode :

1. Let N = length of array A
2. Create array `present` of size $N+1$ and initialize with false
3. For each element x in A :
 If $x > 0$ AND $x \leq N$:
 `present[x] = true`
4. For i from 1 to N :
 If `present[i] == false`:
 return i
5. Return $N + 1$

-Implementation (C):

```
#include <stdio.h>
int findMissing(int A[ ], int N) {
    int present[N+1];

    for(int i = 0; i <= N; i++)
        present[i] = 0;

    for(int i = 0; i < N; i++) {
        if(A[i] > 0 && A[i] <= N)
            present[A[i]] = 1;
    }
    for(int i = 1; i <= N; i++) {
        if(present[i] == 0)
            return i;
    }
    return N + 1;
}
```

-Analysis & Complexity

10-Missing Integer Project

Analysis:

1-Create a helper array present of size $N+1$ and initialize it $\rightarrow$ takes $O(N)$ time.

2-Iterate through the input array A and mark the existing positive elements $\rightarrow O(N)$.

3-Scan the present array from index 1 to N to find the first unmarked position $\rightarrow O(N)$.

4-Return the first index that is not marked as the missing integer.

Step	Operation	count
1	Initialize present array (loop $N+1$ times)	$N+1$
2	Loop through A to mark elements	N
3	Loop to find missing integer	$\leq N$
total		$3N+1 \rightarrow O(N)$

- **Time Complexity:**

- $O(N) + O(N) + O(N) = O(N)$

Second Algorithm (Recursive):

Idea:First sort the array, then recursively scan it while tracking the expected value to find the smallest missing positive number.

10-Missing Integer Project

-Pseudocode :

1. Sort array A
2. Call find(A, index=0, expected=1)

Function find(A, index, expected):

 If $\text{index} \geq \text{length}$:

 return expected

 If $A[\text{index}] == \text{expected}$:

 return find(index+1, expected+1)

 Else if $A[\text{index}] < \text{expected}$:

 return find(index+1, expected)

 Else:

 return expected

-Implementation (C):

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void bubbleSort(int A[], int N) {  
    for (int i = 0; i < N - 1; i++) {  
        for (int j = 0; j < N - i - 1; j++) {  
            if (A[j] > A[j + 1]) {  
                int temp = A[j];  
                A[j] = A[j + 1];  
                A[j + 1] = temp;  
            }  
        }  
    }  
}
```

```
int findRec(int A[], int N, int index, int expected) {  
    if(index >= N)
```

10-Missing Integer Project

```
    return expected;
    if(A[index] == expected)
        return findRec(A, N, index+1, expected+1);

    else if(A[index] < expected)
        return findRec(A, N, index+1, expected);
    else
        return expected;}
int findMissingRecursive(int A[], int N) {
    bubbleSort(A, N);
    return findRec(A, N, 0, 1);}
```

-Analysis & Complexity

Analysis:

1-Sort the array A → takes $O(N^2)$.

2-Start recursive function with:

- index = 0
- expected = 1

3-For each recursive call:

- If element equals expected → move to next index and increment expected
- If element is smaller → skip it
- If element is larger → return expected (missing number found)

4-Continue recursion until the end of the array.

Step	Operation	count
1	Sorting	N^2
2	Recursive calls	$\leq N$

10-Missing Integer Project

total		$N^2 + N \rightarrow O(N^2)$
-------	--	------------------------------

- **Time Complexity:**
 - Sorting takes $O(N^2)$
 - Recursive traversal takes $O(N)$
 - Total = $O(N^2)$

Comparison :

Feature	Recursive	Non-Recursive
Time Complexity	$O(N^2)$	$O(N)$
Speed	Slower	Faster
Implementation	More complex	Simple
Practical Use	For learning	Better choice

-Link:<https://github.com/behery10/Missing-Integer>